

Lab5 Value-Based Reinforcement Learning

Deep Learning, NYCU

朱驛庭, 111550093

1. Introduction

In this lab we implement **Deep Q-Network (DQN)** [3] and several of its sample-efficiency enhancements – **Double DQN (DDQN)** [6], **Prioritized Experience Replay (PER)** [4], **multi-step return** [5], plus **Dueling** [7], **Noisy networks** [1] and **Huber loss** – on two environments of contrasting difficulty:

- **Task 1 – CartPole-v1:** 4-D state, 2 actions, MLP DQN. The environment converges in minutes, so we use it as a sandbox for fast iteration.
- **Task 2 – ALE/Pong-v5 (vanilla DQN):** stacked $84 \times 84 \times 4$ frames, 6 actions, CNN Q-network.
- **Task 3 – ALE/Pong-v5 (enhanced DQN):** vanilla DQN augmented with the six tricks above. The first env-step at which the agent reaches mean reward ≥ 19 is what the spec grades.

Most important findings.

1. Task 1: with the right loss function (MSE in our ablation, see Sec. 3.2), vanilla DQN solves CartPole-v1 perfectly with mean reward 500.00 over the 20 spec seeds.
2. Task 2: tightening the hyper-parameters towards the original Mnih et al. recipe [3] is enough for vanilla CNN-DQN to clear the spec threshold, hitting mean reward 19.65.
3. Task 3: stacking all six tricks on top of the Task 2 baseline pushes the agent across the spec threshold of mean reward ≥ 19 at **1.5M env-steps** (snapshot mean 19.10), earning the 15% sample-efficiency bracket.

Report organisation. Section 2 explains, for each spec-required item, both *how the method works* and

how we implemented it, motivated by the five limitations of vanilla DQN identified in lecture. Section 3 presents the spec-evaluation screenshots, the Task 1 loss ablation, the Task 2 hyper-parameter tightening path, the Task 3 sample-efficiency comparison, and a per-technique ablation discussion.

2. Implementation Details

We refactored the single-file template into a small `src/` package – `src/networks.py` (DQN, NoisyLinear), `src/buffers.py` (ReplayBuffer, PrioritizedReplayBuffer), `src/preprocessor.py`, `src/agent.py` (DQNAgent, also home to the n -step deque), and `src/utils.py`. The driver `dqn.py` stays as the entry point. We deliberately renamed `AtariPreprocessor` to simply `Preprocessor` – the same class transparently handles both the vector observation of CartPole-v1 and the $210 \times 160 \times 3$ pixel observation of Pong-v5, so the `Atari` prefix would be misleading. The other three required class names (DQN, PrioritizedReplayBuffer, DQNAgent) are kept as-is.

2.1. Bellman Error for DQN

The vanilla DQN loss in Eq. (1) of the spec is the squared TD error

$$\mathcal{L}_{\text{DQN}}(\theta) = \frac{1}{2} \left(r + \gamma \max_{a'} Q(s', a'; \bar{\theta}) - Q(s, a; \theta) \right)^2,$$

where $\bar{\theta}$ is the *target network*, frozen for **target-update-frequency** updates and then hard-copied from the online network θ . The Bellman error is computed once per gradient step inside `DQNAgent.train`; the core ten lines are:

```
1 q_values = self.q_net(states).gather(1,
  ↪ actions.unsqueeze(1)).squeeze(1)
2 with torch.no_grad():
3     next_q_values =
  ↪ self.target_net(next_states).max(1)[0]
```

```

5 # Multi-step return
6 gamma_n = self.gamma ** self.n_step
7 target_q_values = returns + gamma_n * (1.0 -
  ↳ dones) * next_q_values
8
9 td_errors = q_values - target_q_values
10 loss = loss_fn(td_errors, weights,
  ↳ self.loss_fn_type, self.huber_beta)

```

Two points worth emphasising. First, the variable `td_errors` is the *sample* Bellman error $\delta_i = Q_\theta(s_i, a_i) - y_i$ on the sampled minibatch – not the true Bellman residual, which would require an expectation over the transition kernel that we do not have access to. The whole DQN training procedure is essentially stochastic gradient descent on the squared sample Bellman error, with the target $y_i = r + \gamma^n \max_{a'} Q_{\bar{\theta}}(s', a') \cdot (1 - d)$ treated as a fixed regression label thanks to `torch.no_grad()`. Second, `returns` is already the n -step bootstrapped return assembled by the agent (Sec. 2.2.6), so the same line of code handles both the 1-step and the multi-step setting via the `gamma_n` factor. The scalar loss is then produced by our own `loss_fn` (Sec. 2.2.2), which folds in the PER importance-sampling weights. With this scaffold in place, Sec. 2.2 layers six well-known enhancements on top, each motivated by a specific limitation of vanilla DQN.

2.2. Six Enhancements over Vanilla DQN

Figure 1 reproduces the lecture slide that motivates Rainbow [2]: it lists five well-known limitations of vanilla DQN, and the table beneath it maps each limitation to the canonical fix. We adopt all five fixes plus a sixth one (n -step return) which complements them.

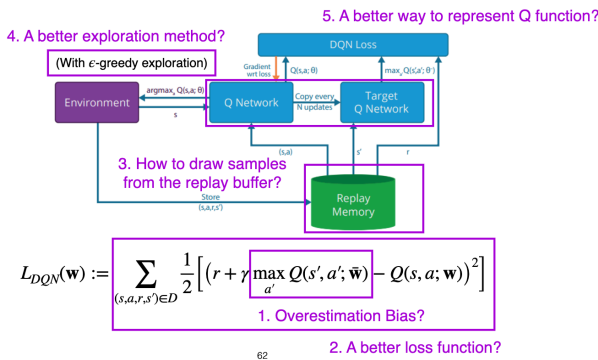


Figure 1. The five limitations of vanilla DQN identified in the lecture slide 20260416-Value-Based-RL.pdf. Each row of Tab. 1 names the limitation and the corresponding trick.

The next paragraphs describe each row – both the concept and our concrete implementation.

Vanilla DQN limitation	Trick
1. Overestimation bias from $\max_a Q$	Double DQN
2. Quadratic loss explodes on large TD errors	Huber loss
3. Uniform replay wastes informative samples	Prioritized Replay
4. ϵ -greedy exploration is too crude	Noisy networks
5. Single-stream Q-net entangles V and A	Dueling networks
(+ slow credit assignment from 1-step bootstrap)	n -step return

Table 1. The six enhancements implemented on top of vanilla DQN in our Task 3 agent. The first five answer the lecture’s five limitations; n -step return additionally accelerates credit assignment, as in Rainbow [2].

2.2.1 Double DQN (limitation 1)

Vanilla DQN uses the same target network both to *select* the best next action and to *evaluate* that action. Because the target uses a max over noisy Q estimates, random positive errors are more likely to be selected, which creates systematic overestimation bias. DDQN [6] fixes this by decoupling the two roles: the online network selects the next action, while the target network evaluates it. Equation (2) of the spec becomes

$$a^*(s') = \arg \max_{a'} Q(s', a'; \theta), \quad \text{tgt} = r + \gamma Q(s', a^*(s'); \bar{\theta}).$$

The change relative to Sec. 2.1 is exactly the `next_actions` line – one extra forward pass through the online net replaces the max operator on the target net:

```

1 q_values = self.q_net(states).gather(1,
  ↳ actions.unsqueeze(1)).squeeze(1)
2
3 with torch.no_grad():
4     # DDQN: use online Q-net to *select* the
  ↳ next action,
5     # target Q-net to *evaluate* it.
6     next_actions =
  ↳ self.q_net(next_states).argmax(dim=1,
  ↳ keepdim=True)
7     next_q_values = self.target_net(next_states)
  ↳ \
  ↳ .gather(1, next_actions).squeeze(1)
8
9
10 # Multi-step return
11 gamma_n = self.gamma ** self.n_step
12 target_q_values = returns + gamma_n * (1.0 -
  ↳ dones) * next_q_values
13
14 td_errors = q_values - target_q_values
15 loss = loss_fn(td_errors, weights,
  ↳ self.loss_fn_type, self.huber_beta)

```

DDQN is enabled with `--double-dqn`; it is on for Task 3 and off for Task 1/2.

2.2.2 Huber Loss (limitation 2)

The vanilla DQN objective uses squared TD error. This is fine when the TD errors are small and well-behaved, but early in Atari training the targets are noisy and a few transitions can have very large $|\delta|$. Since MSE gives a gradient proportional to $|\delta|$, those outliers can dominate an entire minibatch and destabilise learning. Huber loss keeps the quadratic shape near zero but switches to a linear regime for large errors:

$$\mathcal{L}_{\text{Huber}}(\delta) = \begin{cases} \frac{1}{2}\delta^2/\beta & \text{if } |\delta| \leq \beta \\ |\delta| - \frac{\beta}{2} & \text{otherwise.} \end{cases}$$

We write the per-sample loss explicitly rather than calling `torch.nn.functional.huber_loss`, because the same function also needs to fold in the PER importance-sampling weights and support both MSE and Huber through one switch:

```
1 def loss_fn(td_errors, weights, kind, beta=1.0):
2     if kind == "mse":
3         return (weights *
4             ↪ td_errors.pow(2)).mean()
5     if kind == "huber":
6         abs_e = td_errors.abs()
7         per_sample = torch.where(
8             abs_e < beta,
9             0.5 * td_errors.pow(2) / beta,
10            abs_e - 0.5 * beta,
11        )
12    return (weights * per_sample).mean()
13    raise ValueError(f"Invalid loss type:
14    ↪ {kind!r}")
```

We use Huber ($\beta = 1$) for Task 3 (Atari, noisy targets) and MSE for Task 1 (CartPole, smooth and deterministic) – see Sec. 3.2 for the controlled CartPole ablation that motivated this split.

2.2.3 Prioritized Experience Replay (limitation 3)

Uniform replay samples every stored transition with equal probability. This wastes many gradient steps on redundant or already-solved frames, while rare high-error transitions – for example the first successful ball return in Pong – may be replayed too infrequently. PER [4] biases sampling toward transitions with large TD error by using

$$P(i) = \frac{p_i^\alpha}{\sum_j p_j^\alpha}, \quad p_i = |\delta_i| + \epsilon.$$

Because this changes the sampling distribution, we multiply each per-sample loss by the importance-sampling weight $w_i = (N \cdot P(i))^{-\beta} / \max_j w_j$ and anneal β from 0.4 to 1.0. Our implementation is the simple proportional variant: a flat `np.float32` priority array plus `np.random.choice`. It is $O(N)$ per sampling call rather than sum-tree $O(\log N)$, but sampling was not the bottleneck compared with CNN forward/backward, and the simpler code was easier to verify.

```
1 class PrioritizedReplayBuffer:
2     """Proportional Prioritized Experience
3     ↪ Replay (Schaul et al., 2016)."""
4
5     def __init__(self, capacity, alpha=0.6,
6         ↪ beta=0.4,
7             beta_increment=2.5e-6,
8             ↪ eps=1e-6):
9         self.capacity = capacity
10        self.buffer: list[Transition] = []
11        self.pos = 0
12        self.priorities = np.zeros((capacity,),
13            ↪ dtype=np.float32)
14        self.alpha, self.beta = alpha, beta
15        self.beta_increment, self.eps =
16            ↪ beta_increment, eps
17
18    def add(self, transition, error=None):
19        # New transitions get the max priority
20        ↪ seen so far,
21        # so they are guaranteed to be sampled
22        ↪ at least once.
23        if error is None:
24            p = self.priorities.max() if
25                ↪ len(self.buffer) > 0 else 1.0
26        else:
27            p = (abs(error) + self.eps) **
28                ↪ self.alpha
29        if len(self.buffer) < self.capacity:
30            self.buffer.append(transition)
31        else:
32            self.buffer[self.pos] = transition
33            self.priorities[self.pos] = p
34            self.pos = (self.pos + 1) %
35                ↪ self.capacity
36
37    def sample(self, batch_size):
38        N = len(self.buffer)
39        probs = self.priorities[:N] /
40            ↪ self.priorities[:N].sum()
41        indices = np.random.choice(N,
42            ↪ batch_size, p=probs)
43        weights = (N * probs[indices]) **
44            ↪ (-self.beta)
45        weights /= weights.max()
46            ↪ # normalise
47        self.beta = min(1.0, self.beta +
48            ↪ self.beta_increment)
49        batch = [self.buffer[i] for i in
50            ↪ indices]
51        states, actions, rewards, next_states,
52            ↪ dones = zip(*batch)
```

```

36         return states, actions, rewards,
           ↪ next_states, dones, indices, weights
37
38     def update_priorities(self, indices,
           ↪ errors):
39         for idx, err in zip(indices, errors):
40             self.priorities[idx] = (abs(err) +
           ↪ self.eps) ** self.alpha

```

The TD-error magnitudes returned by the agent are fed back via `update_priorities` after every gradient step. We use $\alpha = 0.6$, $\beta_0 = 0.4$ with `beta-increment` $= 2.5 \cdot 10^{-6}$ (so β reaches 1 around 240k updates), and $\epsilon = 10^{-6}$ throughout Task 3.

2.2.4 Noisy Networks (limitation 4)

Vanilla DQN explores via ϵ -greedy: with probability ϵ it ignores the Q-values and picks a uniformly random action. This is simple, but the randomness is state-independent and locally unstable – in Pong, for example, the paddle may jitter up and down on consecutive frames instead of committing to a useful multi-step movement. NoisyNet [1] replaces this action-space randomness with *parameter-space* noise: the policy is still stochastic, but the randomness comes from perturbing the Q-network itself, so action choices remain more coherent and state-dependent.

Concretely, NoisyLinear replaces the parameters (W, b) of `nn.Linear` with

$$W = W_\mu + W_\sigma \odot \varepsilon^W, \quad b = b_\mu + b_\sigma \odot \varepsilon^b,$$

where ε is factorised Gaussian noise and W_σ, b_σ are learned noise scales. Thus the network can automatically reduce noise in well-learned regions and keep it where exploration is still useful. In our implementation, enabling `--noisy-net` replaces the final FC layers with NoisyLinear and sets `epsilon_start = epsilon_min = 0`, because exploration is now handled by the network rather than by random action selection.

```

1 class NoisyLinear(nn.Module):
2     def __init__(self, in_features,
           ↪ out_features, sigma_init=0.5):
3         super().__init__()
4         self.in_features = in_features
5         self.out_features = out_features
6         self.weight_mu = nn.Parameter(torch_
           ↪ .empty(out_features,
           ↪ in_features))
7         self.weight_sigma = nn.Parameter(torch_
           ↪ .empty(out_features,
           ↪ in_features))
8         self.bias_mu = nn.Parameter(torch_
           ↪ .empty(out_features))

```

```

9         self.bias_sigma = nn.Parameter(torch_
           ↪ .empty(out_features))
10        self.register_buffer("weight_epsilon",
           ↪ torch.empty(out_features,
           ↪ in_features))
11        self.register_buffer("bias_epsilon",
           ↪ torch.empty(out_features))
12        self.sigma_init = sigma_init
13        self.reset_parameters()
14        self.reset_noise()
15
16    def reset_parameters(self):
17        mu_range = 1.0 /
           ↪ math.sqrt(self.in_features)
18        self.weight_mu.data.uniform_(-mu_range,
           ↪ mu_range)
19        self.bias_mu.data.uniform_(-mu_range,
           ↪ mu_range)
20        self.weight_sigma.data.fill_(self_
           ↪ .sigma_init /
           ↪ math.sqrt(self.in_features))
21        self.bias_sigma.data.fill_(self_
           ↪ .sigma_init /
           ↪ math.sqrt(self.out_features))
22
23    @staticmethod
24    def _scale_noise(size):
25        x = torch.randn(size)
26        return x.sign().mul_(x.abs().sqrt_())
27
28    def reset_noise(self):
29        eps_in =
           ↪ self._scale_noise(self.in_features)
30        eps_out =
           ↪ self._scale_noise(self.out_features)
31        self.weight_epsilon.copy_(eps_out_
           ↪ .outer(eps_in))
32        self.bias_epsilon.copy_(eps_out)
33
34    def forward(self, x):
35        if self.training:
36            weight = self.weight_mu +
           ↪ self.weight_sigma *
           ↪ self.weight_epsilon
37            bias = self.bias_mu +
           ↪ self.bias_sigma *
           ↪ self.bias_epsilon
38        else:
39            weight, bias = self.weight_mu,
           ↪ self.bias_mu
40        return F.linear(x, weight, bias)

```

2.2.5 Dueling Networks (limitation 5)

DQN estimates a separate $Q(s, a)$ for every action, even though in many states the first-order question is simply whether the state itself is good or bad. In Pong, for example, when the ball is far from the paddle, most actions have almost the same immediate value; what matters more is the state value (are we likely to win this rally?) than the tiny action differences. Dueling networks [7] encode this intuition by splitting $Q(s, a)$

into a state-value stream $V(s)$ and an advantage stream $A(s, a)$: first judge the quality of the state, then judge each action’s relative advantage within that state. The two streams are recombined as

$$Q(s, a) = V(s) + (A(s, a) - \frac{1}{|A|} \sum_{a'} A(s, a')).$$

The mean subtraction normalises the advantage stream and makes the decomposition identifiable: $V(s)$ captures the shared state quality, while $A(s, a) - \text{mean}_{a'} A(s, a')$ captures only the relative preference among actions. In code, `--dueling-dqn` swaps the single FC head for two parallel heads (`Linear` or `NoisyLinear`, depending on `--noisy-net`) and performs the recombination in `forward`:

```
1 def _build_head(self, head_in: int, num_actions:
  ↳ int) -> None:
2     L = self._linear_cls
3     if self._mode == "cnn":
4         stream = lambda out:
5             ↳ nn.Sequential(L(head_in, 512),
6                 ↳ nn.ReLU(), L(512, out))
7     else:
8         stream = lambda out: L(head_in, out)
9
10    if self.dueling:
11        self.value_stream = stream(1)
12        self.advantage_stream =
13            ↳ stream(num_actions)
14    else:
15        self.fc = stream(num_actions)
16
17 def forward(self, x: torch.Tensor) ->
  ↳ torch.Tensor:
18     if self._mode == "cnn":
19         x = x.float() / 255.0
20     features = self.features(x)
21     if self.dueling:
22         value = self.value_stream(features)
23         advantage =
24             ↳ self.advantage_stream(features)
25         return value + (advantage -
26             ↳ advantage.mean(dim=1, keepdim=True))
27     return self.fc(features)
```

2.2.6 n -step Return

The original 1-step DQN target only propagates reward back by one transition per update. This is slow in Pong because the reward is sparse: most frames have reward 0, and the $+1/-1$ signal only appears after a rally ends. n -step return makes the target less myopic by accumulating several real rewards before bootstrapping from the target network. For a transition starting at time t , we store the composite return

$$R_t^{(n)} = \sum_{k=0}^{n-1} \gamma^k r_{t+k}.$$

The Bellman target then becomes $R_t^{(n)} + \gamma^n Q_{\theta}(s_{t+n}, a^*)$ instead of the usual $r_t + \gamma Q_{\theta}(s_{t+1}, a^*)$. In practice, the agent keeps a short deque, emits a composite transition once it has collected n steps, and flushes the remaining tail transitions at the end of an episode:

```
1 def _compute_n_step(self):
2     s0, a0, _, _, _ = self.n_step_buffer[0]
3     _, _, _, s_last, done_last =
4         ↳ self.n_step_buffer[-1]
5     r_n = 0.0
6     s_n, done_n = s_last, done_last
7     for k, (_, _, r, s_next, done) in
8         ↳ enumerate(self.n_step_buffer):
9         r_n += (self.gamma ** k) * r
10        s_n, done_n = s_next, done
11        if done:
12            ↳ truncate at terminal
13            break
14    return s0, a0, r_n, s_n, done_n
15
16 def store_transition(self, s, a, r, s_next,
  ↳ done):
17     self.n_step_buffer.append((s, a, r, s_next,
18         ↳ bool(done)))
19     if len(self.n_step_buffer) < self.n_step:
20         return
21     ↳ warm-up: not enough yet
22     self.memory.add(self._compute_n_step())
23
24 def on_episode_end(self):
25     # Flush the tail so the last (n-1)
26     ↳ transitions are not lost.
27     while len(self.n_step_buffer) > 0:
28         self.memory.add(self._compute_n_step())
29         self.n_step_buffer.popleft()
```

This is exactly why the loss code in Sec. 2.1 uses `gamma_n = self.gamma ** self.n_step`. The trade-off is bias-variance: larger n gives faster credit assignment but noisier targets. We use $n = 3$ for Task 3 as a conservative middle ground.

2.3. W&B Tracking

The original template logged everything into one flat namespace, which quickly became hard to read once we started running parallel ablations. I reorganised the logs into semantic groups and explicitly bound each group to the correct x-axis with `wandb.define_metric`. The resulting dashboard separates environment-step curves from gradient-update curves, so the spec-required plots in Sec. 3 use the intended axis without manual relabelling.

Group	Representative metrics	x-axis
rollout/*	episode return, length, epsilon	env_step
eval/*	20-seed eval reward, best reward	env_step
train/*	TD error, Q statistics, loss, PER β	update_step
global/*	raw counters for env/update steps	—

3. Analysis and Discussions

All numbers reported below follow the spec evaluation protocol (§4.2.2): 20 episodes with seeds $0, 1, \dots, 19$, reporting the mean and standard deviation of undiscounted episode return. The terminal screenshots are collected at the end of the report in Sec. 4; the main analysis keeps only the compact numerical summary and the training curves.

3.1. Spec Evaluation Summary

Task 1 reaches the CartPole cap exactly (mean 500.00 ± 0.00), earning the full 15/15. Task 2 reaches mean Pong reward 19.65 ± 1.46 , above the full-credit threshold. For Task 3, Tab. 2 shows that the 1.5M snapshot is the first checkpoint above mean reward 19, which is the submitted 15/20 sample-efficiency bracket; the **best** checkpoint is numerically identical to that snapshot in our eval.

Snapshot	Mean	≥ 19 ?	Bracket
0.6M	17.10	no	—
1.0M	17.45	no	—
1.5M	19.10	yes	15/20
2.0M	18.25	no	—
2.5M	18.60	no	—
best	19.10	yes	—

Table 2. Spec evaluation of every Task 3 sample-efficiency snapshot. The earliest snapshot crossing mean reward ≥ 19 is the 1.5M one, which earns the 15% bracket on the spec grading table. The **best** checkpoint is the 1.5M snapshot.

3.2. Task 1: CartPole Loss Ablation

CartPole trains in ~ 10 min, so it is the right environment to study *which loss* fits a small low-noise problem before moving to Pong. With every other hyperparameter pinned to the baseline of Tab. 3, we ablate four settings: MSE and Huber with $\beta \in \{1.0, 2.5, 5.0\}$. Figure 2 is also the Task 1 training curve required by the spec, with the x-axis set to `global/env_step`.

Finding. MSE wins on this low-noise environment: it is the only setting whose evaluation reward consistently reaches 500, and it does so faster than the Hu-

Hyper-parameter	Value
Episodes (cap)	1500
Batch size	64
Learning rate	1×10^{-4}
Replay buffer	uniform, 100 000
Replay start	5 000
Target update freq.	1 000 env-steps
ε schedule	exp. $1.0 \rightarrow 0.05$ ($\lambda=0.99995$)
Discount γ	0.99
Eval	every 10 episodes, 20 seeds
Early stop	mean reward ≥ 480 for 3 evals
Seed	42

Table 3. CartPole-v1 baseline shared across the loss ablation; only **loss-fn-type** and **huber-beta** are swept.

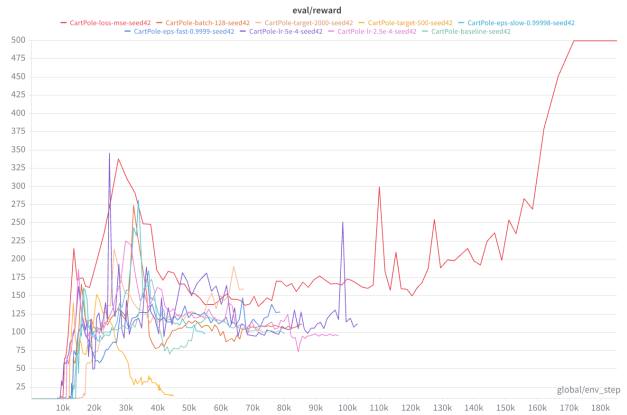


Figure 2. **Task 1** – `eval/reward` vs. environment step for the four CartPole-v1 loss settings. The submitted run (`loss-mse / seed42`, blue) is the only one that reaches and stays at the maximum reward of 500.

ber variants. Huber is useful later on Atari because it clips large TD-error outliers, but in deterministic CartPole those clipped gradients mostly slow down learning without providing much stability benefit. This is why the submitted Task 1 checkpoint uses MSE.

3.3. Task 2: From Untuned Vanilla Pong to Mnih-Style DQN

Our first Pong run (orange in Fig. 3) used hyperparameters mostly inherited from CartPole: larger replay memory, target update every 10 000 env-steps, a slow exponential ε decay, and no explicit train-frequency. It learned, but sample efficiency was poor. Tightening the setup toward the original Mnih et al. recipe [3] gives the blue run, **mnih-standard**, which is still vanilla CNN-DQN but trains much faster and becomes the submitted Task 2 model.

The two Task 2 changes that matter most are: (i) targeting every 1 000 *gradient updates* instead of every

Hyper-param.	Untuned	Mnih-style
Memory size	200 000	100 000
Replay start	50 000	10 000
Target update freq.	10 000 env-steps	1 000 <i>updates</i>
Train frequency	every env-step	every 4 env-steps
ϵ schedule	exp. $\lambda=10^{-7}$	linear 1.0 $\rightarrow$ 0.01 over 10^6 steps
Loss	MSE	MSE

Table 4. Task 2 hyper-parameter tightening: both columns are vanilla CNN-DQN; only the right column is the submitted Task 2 configuration.

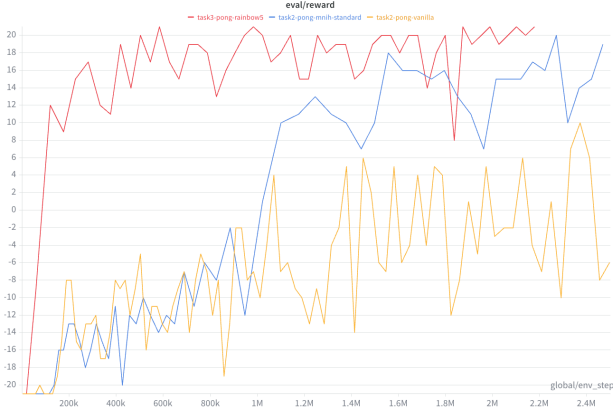


Figure 3. **Pong sample efficiency** – orange is the untuned vanilla Pong run, blue is the Mnih-style vanilla DQN submitted for Task 2, and red is the fully enhanced Task 3 agent. Hyper-parameter tightening alone greatly accelerates Task 2, while the Task 3 enhancements push the curve above the mean-reward-19 threshold by 1.5M env-steps.

10 000 env-steps, which keeps the bootstrap target close enough to the online network that Q-values can grow; and (ii) decaying ϵ linearly to 0.01 over 1M env-steps, which lets policy improvement translate into rollout return instead of continuing to inject too much random action noise.

3.4. Task 3: Enhanced DQN Sample Efficiency

The Task 2 `mnih-standard` run and Task 3 `rainbow5` run share batch size, optimiser, learning rate, and frame stack; Tab. 5 lists only the important differences. This makes the blue-vs-red comparison in Fig. 3 a clean read on the effect of adding DDQN, PER, Huber, NoisyNet, Dueling, and n -step return.

The enhanced agent is dramatically more sample-efficient: at 1.5M env-steps it has already reached spec mean reward 19.10, while the vanilla agent at the same env-step is still around -10 to 0 on `eval/reward`. The vanilla agent only catches up around the 4–5M env-step range. Concretely, the enhanced agent crosses the spec threshold roughly **3–4 $\times$** earlier in env-steps than

Hyper-param.	Vanilla (T2)	Enhanced (T3)
Replay buffer	uniform	PER ($\alpha=0.6$, $\beta=0.4 \rightarrow 1$)
n -step return	1	3
Bootstrap target	vanilla DQN	Double DQN
Network head	single Q-stream	Dueling V/A streams
Exploration	ϵ -greedy linear $1 \rightarrow 0.01$	$\epsilon=0$, NoisyNet
Loss	MSE	Huber ($\beta=1$)
Memory size	100 000	50 000
Target update freq.	1 000 updates	500 updates
Train freq. / per-step	every 4 env-steps, 1	every 4 env-steps, 4

Table 5. Vanilla DQN (Task 2) vs. enhanced DQN (Task 3) hyper-parameters. The bold rows correspond exactly to the six tricks of Tab. 1.

the vanilla one.

3.5. Per-Technique Ablation

To isolate the effect of the three techniques originally required by the spec, we trained technique-only Pong runs on top of the same Mnih-style baseline: DDQN only, PER only, and n -step return only ($n = 3$). We also trained an `all3` run that combines exactly these three techniques (PER + DDQN + n -step). This ablation intentionally does *not* compare Huber, NoisyNet, or Dueling, so we do not draw conclusions about them here.

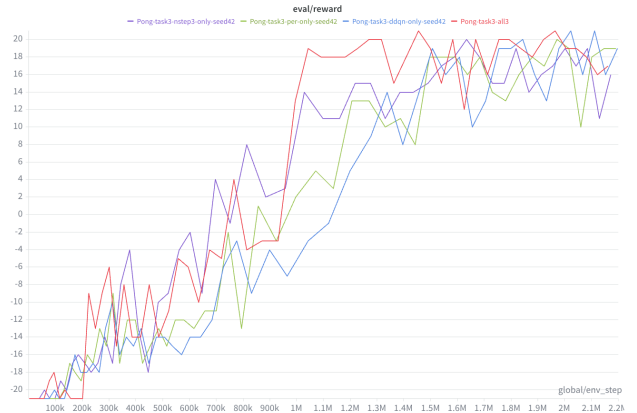


Figure 4. Technique-only ablation on Pong-v5. Purple, green, and blue enable only n -step return, PER, or DDQN respectively; red combines all three. Here “full” means PER+DDQN+ n -step3, not the complete Task 3 stack with Dueling/NoisyNet/Huber.

n -step return. The purple curve improves earliest among the single-technique runs, reaching positive reward before the PER-only and DDQN-only curves. This matches the intuition that Pong’s reward is sparse: aggregating three real rewards before

bootstrapping propagates the rally outcome back faster than pure 1-step TD learning.

PER. The green curve improves more gradually, but becomes competitive after roughly 1.5M env-steps. PER helps once the replay buffer contains informative high-error transitions, because those rare ball-contact / rally-ending transitions are replayed more often than under uniform sampling.

DDQN. The blue curve is the slowest early on, but catches up later and reaches high reward in the final phase. This suggests that DDQN’s main benefit in this setting is not early exploration speed, but more stable bootstrap targets after the policy has started finding useful Pong behaviours.

Combined effect. The red a113 curve is best in sample efficiency: it jumps to high reward around 1.0M env-steps and stays mostly in the 18–21 range afterward. The single-technique curves eventually approach strong performance, but the combination reaches it earlier and with fewer long low-reward phases, showing that the three methods are complementary rather than redundant.

4. Spec-Eval Screenshots

For completeness, this appendix-style section records the terminal outputs used to verify the submitted checkpoints. The filenames are omitted from captions to avoid visually noisy long monospace strings; all submitted checkpoints follow the required LAB5_111550093_*.pt naming convention.

```
(.venv) (base) chueating@gpu10:/project3/chueating/DLP/Lab5$ bash run_evaluation.sh task1
=====
Task 1 (CartPole-v1, vanilla DQN)
=====
ep 00 (seed=0): reward=+500.0
ep 01 (seed=1): reward=+500.0
ep 02 (seed=2): reward=+500.0
ep 03 (seed=3): reward=+500.0
ep 04 (seed=4): reward=+500.0
ep 05 (seed=5): reward=+500.0
ep 06 (seed=6): reward=+500.0
ep 07 (seed=7): reward=+500.0
ep 08 (seed=8): reward=+500.0
ep 09 (seed=9): reward=+500.0
ep 10 (seed=10): reward=+500.0
ep 11 (seed=11): reward=+500.0
ep 12 (seed=12): reward=+500.0
ep 13 (seed=13): reward=+500.0
ep 14 (seed=14): reward=+500.0
ep 15 (seed=15): reward=+500.0
ep 16 (seed=16): reward=+500.0
ep 17 (seed=17): reward=+500.0
ep 18 (seed=18): reward=+500.0
ep 19 (seed=19): reward=+500.0

Episodes: 20
Mean reward: +500.00 (std=0.00, min=+500.0, max=+500.0)
Task 1 score: 15.00 / 15.00 %
```

Figure 5. Task 1 eval screenshot: mean 500.00 ± 0.00 , score 15/15.

```
(.venv) (base) chueating@gpu10:/project3/chueating/DLP/Lab5$ bash run_evaluation.sh task2
=====
Task 2 (ALE/Pong-v5, vanilla DQN with CNN)
=====
A.L.E. Arcade Learning Environment (version 0.11.2+ecc1138)
[Powered by Stella]
ep 00 (seed=0): reward=+19.0
ep 01 (seed=1): reward=+20.0
ep 02 (seed=2): reward=+21.0
ep 03 (seed=3): reward=+20.0
ep 04 (seed=4): reward=+20.0
ep 05 (seed=5): reward=+17.0
ep 06 (seed=6): reward=+21.0
ep 07 (seed=7): reward=+21.0
ep 08 (seed=8): reward=+21.0
ep 09 (seed=9): reward=+17.0
ep 10 (seed=10): reward=+21.0
ep 11 (seed=11): reward=+21.0
ep 12 (seed=12): reward=+18.0
ep 13 (seed=13): reward=+20.0
ep 14 (seed=14): reward=+21.0
ep 15 (seed=15): reward=+20.0
ep 16 (seed=16): reward=+19.0
ep 17 (seed=17): reward=+18.0
ep 18 (seed=18): reward=+21.0
ep 19 (seed=19): reward=+17.0

Episodes: 20
Mean reward: +19.65 (std=1.46, min=+17.0, max=+21.0)
Task 2 score: 20.00 / 20.00 %
```

Figure 6. Task 2 eval screenshot: mean 19.65 ± 1.46 , score 20/20.

```
(.venv) (base) chueating@gpu10:/project3/chueating/DLP/Lab5$ bash run_evaluation.sh task3
=====
Task 3 (ALE/Pong-v5, enhanced DQN: DQN + NoisyNet + DDQN + PER + n-steps)
=====
A.L.E. Arcade Learning Environment (version 0.11.2+ecc1138)
[Powered by Stella]
ep 00 (seed=0): reward=+18.0
ep 01 (seed=1): reward=+18.0
ep 02 (seed=2): reward=+18.0
ep 03 (seed=3): reward=+18.0
ep 04 (seed=4): reward=+18.0
ep 05 (seed=5): reward=+18.0
ep 06 (seed=6): reward=+18.0
ep 07 (seed=7): reward=+18.0
ep 08 (seed=8): reward=+18.0
ep 09 (seed=9): reward=+18.0
ep 10 (seed=10): reward=+18.0
ep 11 (seed=11): reward=+18.0
ep 12 (seed=12): reward=+18.0
ep 13 (seed=13): reward=+18.0
ep 14 (seed=14): reward=+18.0
ep 15 (seed=15): reward=+18.0
ep 16 (seed=16): reward=+18.0
ep 17 (seed=17): reward=+18.0
ep 18 (seed=18): reward=+18.0
ep 19 (seed=19): reward=+18.0

Episodes: 20
Mean reward: +17.65 (std=1.94, min=+13.0, max=+21.0)
Task 3 score: 197.00
```

(a) 0.6M

```
(.venv) (base) chueating@gpu10:/project3/chueating/DLP/Lab5$ bash run_evaluation.sh task3
=====
Task 3 (ALE/Pong-v5, enhanced DQN: DQN + NoisyNet + DDQN + PER + n-steps)
=====
A.L.E. Arcade Learning Environment (version 0.11.2+ecc1138)
[Powered by Stella]
ep 00 (seed=0): reward=+18.0
ep 01 (seed=1): reward=+18.0
ep 02 (seed=2): reward=+18.0
ep 03 (seed=3): reward=+18.0
ep 04 (seed=4): reward=+18.0
ep 05 (seed=5): reward=+18.0
ep 06 (seed=6): reward=+18.0
ep 07 (seed=7): reward=+18.0
ep 08 (seed=8): reward=+18.0
ep 09 (seed=9): reward=+18.0
ep 10 (seed=10): reward=+18.0
ep 11 (seed=11): reward=+18.0
ep 12 (seed=12): reward=+18.0
ep 13 (seed=13): reward=+18.0
ep 14 (seed=14): reward=+18.0
ep 15 (seed=15): reward=+18.0
ep 16 (seed=16): reward=+18.0
ep 17 (seed=17): reward=+18.0
ep 18 (seed=18): reward=+18.0
ep 19 (seed=19): reward=+18.0

Episodes: 20
Mean reward: +17.65 (std=1.94, min=+13.0, max=+21.0)
Task 3 score: 197.00
```

(b) 1.0M

```
(.venv) (base) chueating@gpu10:/project3/chueating/DLP/Lab5$ bash run_evaluation.sh task3
=====
Task 3 (ALE/Pong-v5, enhanced DQN: DQN + NoisyNet + DDQN + PER + n-steps)
=====
A.L.E. Arcade Learning Environment (version 0.11.2+ecc1138)
[Powered by Stella]
ep 00 (seed=0): reward=+18.0
ep 01 (seed=1): reward=+18.0
ep 02 (seed=2): reward=+18.0
ep 03 (seed=3): reward=+18.0
ep 04 (seed=4): reward=+18.0
ep 05 (seed=5): reward=+18.0
ep 06 (seed=6): reward=+18.0
ep 07 (seed=7): reward=+18.0
ep 08 (seed=8): reward=+18.0
ep 09 (seed=9): reward=+18.0
ep 10 (seed=10): reward=+18.0
ep 11 (seed=11): reward=+18.0
ep 12 (seed=12): reward=+18.0
ep 13 (seed=13): reward=+18.0
ep 14 (seed=14): reward=+18.0
ep 15 (seed=15): reward=+18.0
ep 16 (seed=16): reward=+18.0
ep 17 (seed=17): reward=+18.0
ep 18 (seed=18): reward=+18.0
ep 19 (seed=19): reward=+18.0

Episodes: 20
Mean reward: +17.65 (std=1.94, min=+13.0, max=+21.0)
Task 3 score: 197.00
```

(c) 1.5M

```
(.venv) (base) chueating@gpu10:/project3/chueating/DLP/Lab5$ bash run_evaluation.sh task3
=====
Task 3 (ALE/Pong-v5, enhanced DQN: DQN + NoisyNet + DDQN + PER + n-steps)
=====
A.L.E. Arcade Learning Environment (version 0.11.2+ecc1138)
[Powered by Stella]
ep 00 (seed=0): reward=+18.0
ep 01 (seed=1): reward=+18.0
ep 02 (seed=2): reward=+18.0
ep 03 (seed=3): reward=+18.0
ep 04 (seed=4): reward=+18.0
ep 05 (seed=5): reward=+18.0
ep 06 (seed=6): reward=+18.0
ep 07 (seed=7): reward=+18.0
ep 08 (seed=8): reward=+18.0
ep 09 (seed=9): reward=+18.0
ep 10 (seed=10): reward=+18.0
ep 11 (seed=11): reward=+18.0
ep 12 (seed=12): reward=+18.0
ep 13 (seed=13): reward=+18.0
ep 14 (seed=14): reward=+18.0
ep 15 (seed=15): reward=+18.0
ep 16 (seed=16): reward=+18.0
ep 17 (seed=17): reward=+18.0
ep 18 (seed=18): reward=+18.0
ep 19 (seed=19): reward=+18.0

Episodes: 20
Mean reward: +17.65 (std=1.94, min=+13.0, max=+21.0)
Task 3 score: 197.00
```

(d) 2.5M

```
(.venv) (base) chueating@gpu10:/project3/chueating/DLP/Lab5$ bash run_evaluation.sh task3
=====
Task 3 (ALE/Pong-v5, enhanced DQN: DQN + NoisyNet + DDQN + PER + n-steps)
=====
A.L.E. Arcade Learning Environment (version 0.11.2+ecc1138)
[Powered by Stella]
ep 00 (seed=0): reward=+18.0
ep 01 (seed=1): reward=+18.0
ep 02 (seed=2): reward=+18.0
ep 03 (seed=3): reward=+18.0
ep 04 (seed=4): reward=+18.0
ep 05 (seed=5): reward=+18.0
ep 06 (seed=6): reward=+18.0
ep 07 (seed=7): reward=+18.0
ep 08 (seed=8): reward=+18.0
ep 09 (seed=9): reward=+18.0
ep 10 (seed=10): reward=+18.0
ep 11 (seed=11): reward=+18.0
ep 12 (seed=12): reward=+18.0
ep 13 (seed=13): reward=+18.0
ep 14 (seed=14): reward=+18.0
ep 15 (seed=15): reward=+18.0
ep 16 (seed=16): reward=+18.0
ep 17 (seed=17): reward=+18.0
ep 18 (seed=18): reward=+18.0
ep 19 (seed=19): reward=+18.0

Episodes: 20
Mean reward: +17.65 (std=1.94, min=+13.0, max=+21.0)
Task 3 score: 197.00
```

(e) best

Figure 7. Task 3 eval screenshots for sample-efficiency snapshots and best checkpoint.

References

- [1] Meire Fortunato, Mohammad Gheshlaghi Azar, Bilal Piot, Jacob Menick, Ian Osband, Alex Graves, Vlad Mnih, Rémi Munos, Demis Hassabis, Olivier Pietquin, Charles Blundell, and Shane Legg. Noisy networks for exploration. In *ICLR*, 2018. 1, 4
- [2] Matteo Hessel, Joseph Modayil, Hado van Hasselt, Tom Schaul, Georg Ostrovski, Will Dabney, Dan Horgan, Bilal Piot, Mohammad Gheshlaghi Azar, and David Sil-

- ver. Rainbow: Combining improvements in deep reinforcement learning. In *AAAI*, 2018. 2
- [3] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, Stig Petersen, Charles Beattie, Amir Sadik, Ioannis Antonoglou, Helen King, Dharmashan Kumar, Daan Wierstra, Shane Legg, and Demis Hassabis. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015. 1, 6
- [4] Tom Schaul, John Quan, Ioannis Antonoglou, and David Silver. Prioritized experience replay. In *ICLR*, 2016. 1, 3
- [5] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2nd edition, 2018. 1
- [6] Hado van Hasselt, Arthur Guez, and David Silver. Deep reinforcement learning with double Q-learning. In *AAAI*, 2016. 1, 2
- [7] Ziyu Wang, Tom Schaul, Matteo Hessel, Hado van Hasselt, Marc Lanctot, and Nando de Freitas. Dueling network architectures for deep reinforcement learning. In *ICML*, 2016. 1, 4